第 3 课：线性回归模型

吴恩达《Machine Learning Specialization》· Course 1 · Week 1 · Lesson 3

承上：第 2 课我们知道监督学习 = "有答案的练习册"。 **这一课**：请出机器学习里**第一个真正的主角公式**：

$$f(x) = w \cdot x + b$$

看不懂没关系！**这一个笔记本会把它逐字拆开讲透**，完全是零基础。

配套：如果某个数学符号让你卡住，翻一翻旁边的 **《数学基础急救包》**，那里把"变量、函数、平方、Σ、直线方程"都从零讲了一遍。

怎么用：从头往下读，代码格按 `Shift + Enter` 运行。

第 0 步：环境自检

运行下面这格，确认画图工具就绪、中文字体也设好。

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import font_manager
# ---- 选择你电脑上存在的中文字体：Windows 用微软雅黑，Mac 用苹方，Linux 用 Noto ----
_candidates = ["Microsoft YaHei", "SimHei", "PingFang SC",
               "Noto Sans CJK JP", "Noto Sans CJK SC", "Droid Sans Fallback"]
_available = {f.name for f in font_manager.fontManager.ttflist}
_chosen = [c for c in _candidates if c in _available] or ["DejaVu Sans"]
plt.rcParams["font.sans-serif"] = _chosen
plt.rcParams["axes.unicode_minus"] = False
print("本次使用中文字体：", _chosen)
print(" 环境就绪，可以继续啦！")
```

```
本次使用中文字体： ['Noto Sans CJK JP', 'Droid Sans Fallback']
 环境就绪，可以继续啦！
```

一、回顾：还是那 9 套房子

第 2 课末尾那份房价数据——每套房子一个点，横轴面积、纵轴房价。你发现没，这些点**大致排成一条向上的直线**。

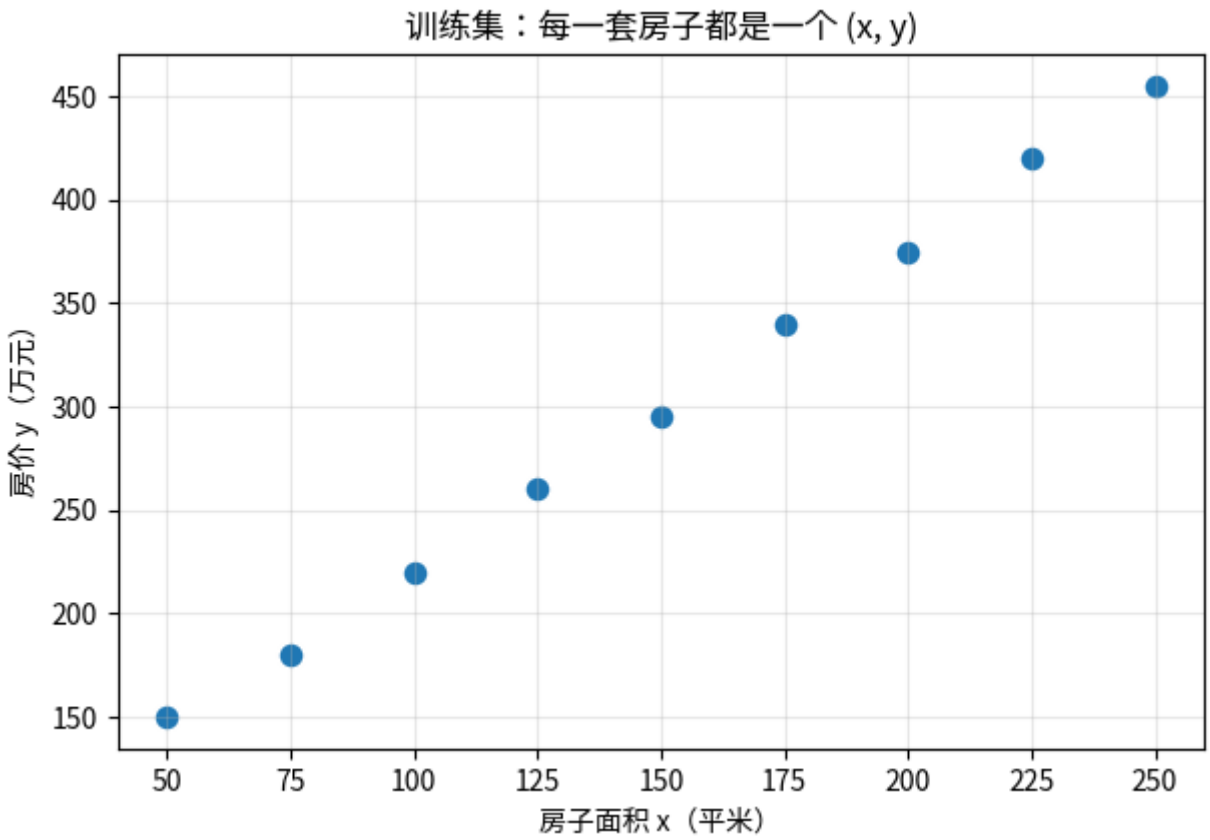
那我们自然会想：**能不能画一条线，尽量贴着这些点？** 有了这条线，以后来一套新房，横着一看就知道大概卖多少钱。

这条线，就是我们要找的"模型"。 先把它画出来看看：

In [2]:

```
# ===== 训练集：9 套房子（和第 2 课同一份数据）=====
x = np.array([50, 75, 100, 125, 150, 175, 200, 225, 250])   # 面积 x
y = np.array([150, 180, 220, 260, 295, 340, 375, 420, 455]) # 房价 y

plt.figure(figsize=(7, 4.5))
plt.scatter(x, y, s=90, color="tab:blue", edgecolor="white", linewidth=1.2)
plt.xlabel("房子面积 x（平米）")
plt.ylabel("房价 y（万元）")
plt.title("训练集：每一套房子都是一个 (x, y)")
plt.grid(alpha=0.3)
plt.show()
```



二、主角登场：直线模型 f(x) = w·x + b

我们请出一条直线来当这个模型，写法是：

$$f(x) = w \cdot x + b$$

先别被它吓到，把每个符号一个一个拆开看：

符号	中文名	它在这里干嘛
`f`	模型 / 函数	就是"预测机器"——丢进面积，吐出预测的房价
`x`	输入	房子的面积（已知）
`f(x)`	预测值	机器**估出来的**房价（注意：不是真实房价！）
`w`	权重（斜率）	控制线"陡不陡"
`b`	偏置（截距）	控制线"高还是低"

用一句人话读出来：

> "预测的房价 `f(x)` = **w** 乘以 面积 `x`，再加上 **b**。"

就这么简单，一段"乘法 + 加法"。

关键理解：`w` 和 `b` 是两个**可以调节的旋钮**——拧 `w` 直线变陡/变缓，拧 `b` 直线整体抬高/降低。

而**"训练机器学习模型"这件事，说白了就是：自动找出最合适的 w 和 b**，让这条线尽量贴着数据。下面我们亲手拧一拧。

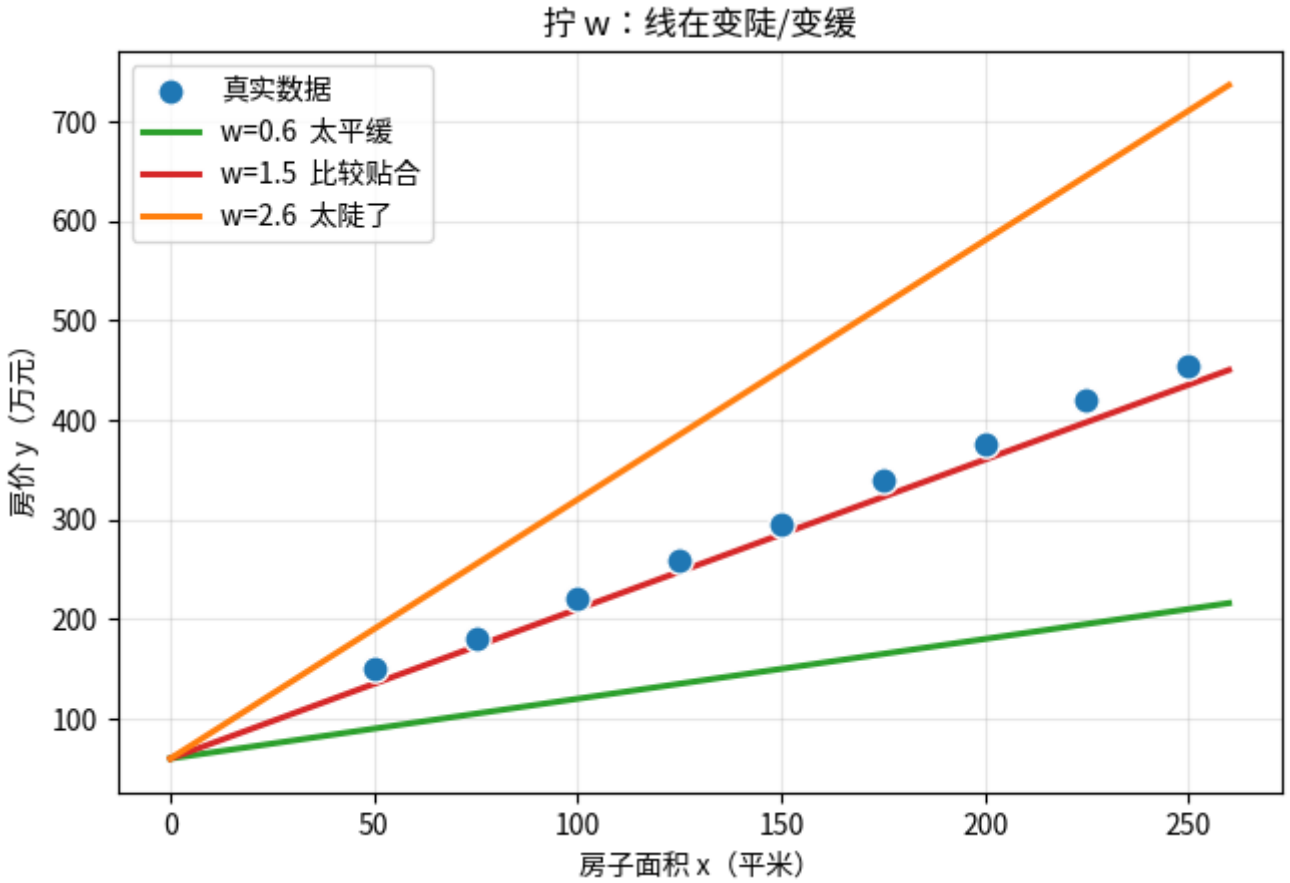
2.1 拧旋钮 w：控制陡不陡

固定 `b` 不动，只改变 `w`，看三条线的区别：

In [3]:

```
def draw_line(w, b, color, label):
    xs = np.linspace(0, 260, 100)      # 横坐标从 0 到 260
    ys = w * xs + b                    # 就是 f(x) = wx + b
    plt.plot(xs, ys, color=color, linewidth=2.2, label=label)

plt.figure(figsize=(7.5, 4.8))
plt.scatter(x, y, s=90, color="tab:blue", edgecolor="white", label="真实数据", zorder=3)
draw_line(0.6, 60, "tab:green",  "w=0.6  太平缓")
draw_line(1.5, 60, "tab:red",    "w=1.5  比较贴合")
draw_line(2.6, 60, "tab:orange", "w=2.6  太陡了")
plt.xlabel("房子面积 x（平米）"); plt.ylabel("房价 y（万元）")
plt.title("拧 w：线在变陡/变缓")
plt.legend(); plt.grid(alpha=0.3)
plt.show()
```



看：三条线**起点高度一样**（b 都是 60），但倾斜程度不同。绿色太平、橙色太陡、红色比较贴合。

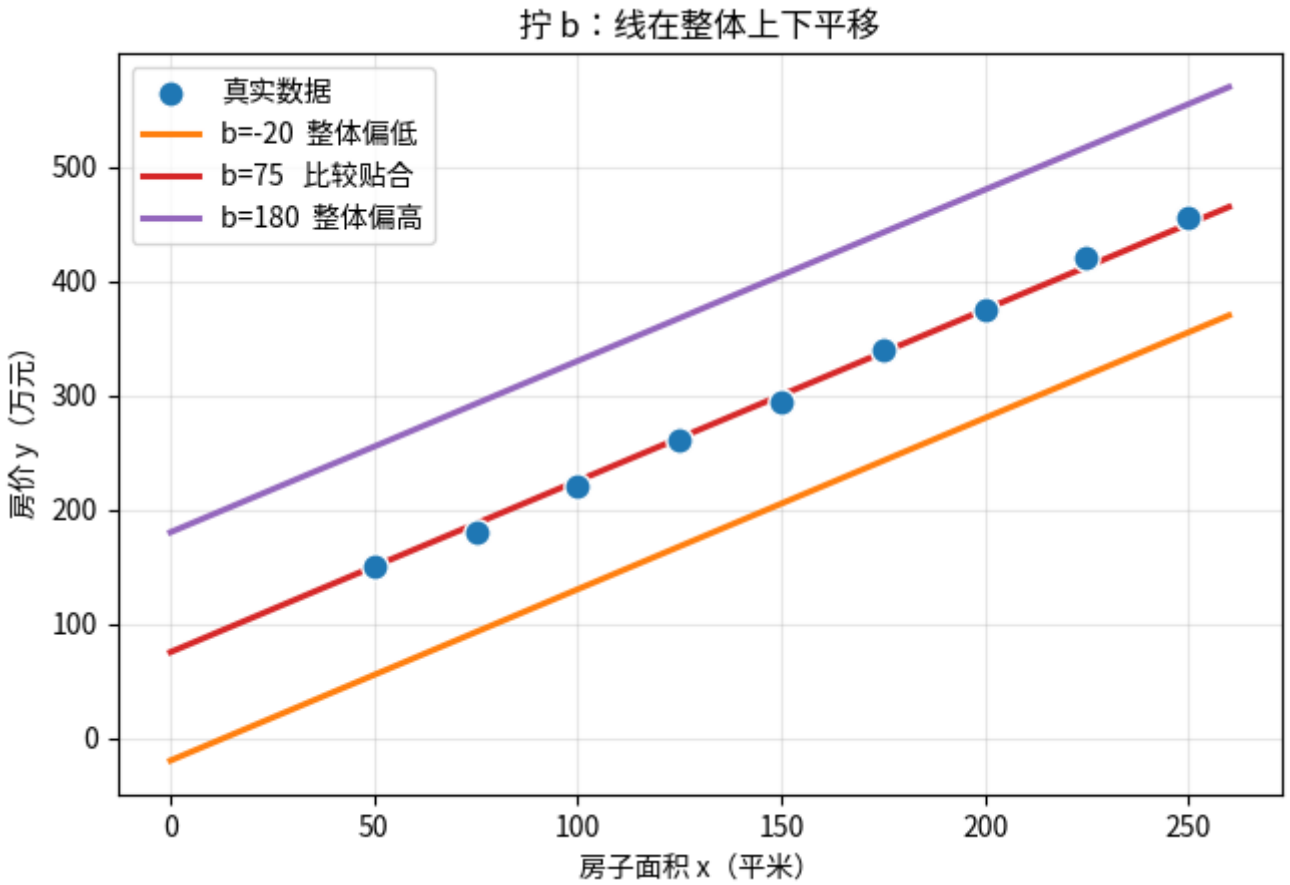
记住：w 越大，线越陡。 它管的是"面积每多 1 平米，房价大约涨多少"。

2.2 拧旋钮 b：控制高还是低

这次固定 `w`，只改变 `b`：

In [4]:

```
plt.figure(figsize=(7.5, 4.8))
plt.scatter(x, y, s=90, color="tab:blue", edgecolor="white", label="真实数据", zorder=3)
draw_line(1.5, -20, "tab:orange", "b=-20  整体偏低")
draw_line(1.5,  75, "tab:red",    "b=75   比较贴合")
draw_line(1.5, 180, "tab:purple", "b=180  整体偏高")
plt.xlabel("房子面积 x（平米）"); plt.ylabel("房价 y（万元）")
plt.title("拧 b：线在整体上下平移")
plt.legend(); plt.grid(alpha=0.3)
plt.show()
```



看：三条线**倾斜一模一样**（w 都是 1.5），但整体位置不同——b 大的往上抬，b 小的往下沉。

记住：b 管的是"线的整体高低"。

> 到这里你已经抓住了线性回归的全部灵魂：**训练 = 找到一对最合适的 (w, b)，让线最贴合。** 那问题来了——**怎么才算"贴合"？** 下一节见分晓。

三、"哪条线最好？"—— 先认识"误差"

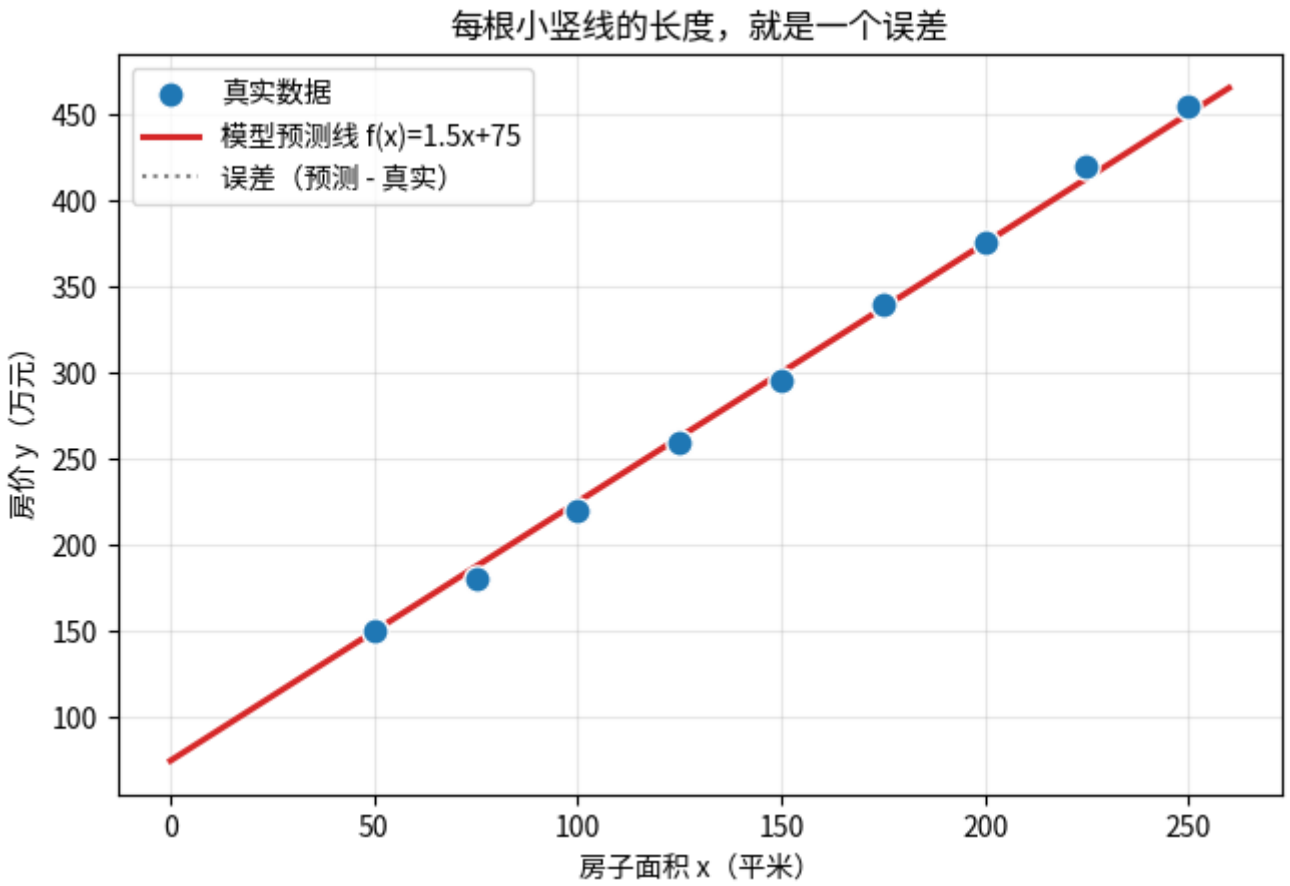
要挑最好的线，得先有一把"尺子"去量每条线**错得有多离谱**。这把尺子叫 **误差（error）**：对每个点，量一量"线的预测值"和"真实值"差了多远。

$$\text{误差} = f(x) - y = (\text{预测的房价}) - (\text{真实的房价})$$

我们把每个点垂直连到线上，那段"小棍子"的长度，就是误差：

In [5]:

```
plt.figure(figsize=(7.5, 4.8))
plt.scatter(x, y, s=90, color="tab:blue", edgecolor="white", label="真实数据", zorder=3)
w, b = 1.5, 75                          # 用刚才那条"比较贴合"的线
xs = np.linspace(0, 260, 100)
plt.plot(xs, w*xs + b, color="tab:red", linewidth=2.2, label="模型预测线 f(x)=1.5x+75")
for xi, yi in zip(x, y):                # 每个点的误差（真实点连到线上的竖线）
    plt.plot([xi, xi], [yi, w*xi + b], color="gray", linestyle=":", linewidth=1.8)
plt.plot([], [], color="gray", linestyle=":", label="误差（预测 - 真实）")
plt.xlabel("房子面积 x（平米）"); plt.ylabel("房价 y（万元）")
plt.title("每根小竖线的长度，就是一个误差")
plt.legend(); plt.grid(alpha=0.3)
plt.show()
```



看：每根灰色小竖线，就是那套房子的"预测错了多少"。线贴得越紧，小棍子越短；所有小棍子都短，这条线就是"好线"。

机器学习要做的，就是把所有误差"加起来"（还记得 Σ 吗），让这个总和尽可能小。 这个"误差总和"有专门的名字，叫**成本函数（Cost Function）**，通常记作 `J(w, b)`。

> **这就是第 4 课的主角。** 这一课只要先建立直觉：**"训练模型" = "调 w 和 b，把总误差压到最小"**。

（好奇的话，成本函数大概长这样——先混个脸熟，不用现在懂：）

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f(x^{(i)}) - y^{(i)}\right)^2$$

四、小结：这一课你收获了

1. **线性回归模型**：`f(x) = w·x + b`，一条**可以调陡度(w)、调高低(b)的直线**。
2. **两个旋钮**：`w` 管陡不陡，`b` 管高不高。
3. **训练的本质**：自动找到最合适的 `w` 和 `b`，让**误差总和**最小。
4. **下一站**：用**成本函数 `J(w,b)`** 精确衡量"误差总和"。

五、反转课堂 · 课后作业

1. 用你自己的话解释：`f(x) = wx + b` 里，`w` 和 `b` **分别**在控制什么？
2. 如果 `w` 调得**非常大**，直线会变成什么样？`b` 调得**非常大**呢？
3. 面积 130 平米、真实房价 270 万。若模型是 `f(x)=1.5x+75`，**预测房价**是多少？**误差**是正还是负？

哪里卡住，随时来找我，我们一个符号一个符号地拆

下一站

下一课（第 4 课）：成本函数 `J(w, b)` ——我们会把这个"误差总和"变成一个精确的公式，看看怎么用它自动找出最好的 `w` 和 `b`。

> 编号说明：第 4 课**依然属于 Lesson 3 这一课**——线性回归的知识点很多，我们会一课一课往下拆。